
THREATFOREST: Multi-Agent Attack Tree Generation with MITRE ATT&CK Mapping

Cristian Leo
Amazon Web Services
crisleoo@amazon.com

Anton Dykyi
Amazon Web Services
antondk@amazon.co.uk

Danny Cortegaca
Amazon Web Services
dicorteg@amazon.co.uk

Daniel Begimher
Amazon Web Services
dbbegimh@amazon.com

Abstract

Threat modeling is essential for secure software development, yet manual analysis of cloud-native architectures is time-consuming and requires scarce security expertise. We present THREATFOREST, a multi-agent system that automatically generates structured attack trees from source code repositories, maps attack steps to MITRE ATT&CK techniques, and synthesizes actionable mitigations. THREATFOREST decomposes threat modeling into a six-stage pipeline—repository analysis, threat generation, attack tree construction, TTP mapping, mitigation synthesis, and report generation—orchestrated as a directed graph with deterministic verification gates. A key technical contribution is our hybrid TTP mapping approach: domain-specific embeddings (ATTACK-BERT) retrieve candidate techniques via cosine similarity, followed by an LLM reviewer that refines mappings with access to top- K alternatives. We evaluate THREATFOREST across **[TODO: N]** application domains using 16 scoring dimensions spanning four capabilities, supported by an observability infrastructure built on OpenTelemetry and Langfuse that enables both automated metrics and subject matter expert (SME) review. **[TODO: Baseline results summary.]** To our knowledge, THREATFOREST is the first end-to-end system that takes a code repository as input and produces MITRE ATT&CK-mapped attack trees with evidence-based mitigations.

1 Introduction

Threat modeling is a cornerstone of secure software development [2], requiring practitioners to systematically identify threats, analyze attack vectors, and propose mitigations. As cloud-native architectures grow in complexity—a single system may span dozens of managed services, each with distinct access control models, encryption configurations, and network boundaries—manual threat modeling becomes increasingly impractical. Industry surveys consistently report that organizations either skip threat modeling entirely or perform it only for the most critical systems, leaving significant attack surface unanalyzed.

Existing tools address fragments of this problem. Microsoft Threat Modeling Tool [17] and OWASP Threat Dragon [16] provide structured diagramming but automate little of the core reasoning. AWS Threat Composer [18] offers guided threat statement authoring but requires manual input for each threat. Recent work has explored using large language models (LLMs) for threat identification [8, 9],

but these approaches typically stop at threat enumeration—they do not produce structured attack trees, map to standardized frameworks like MITRE ATT&CK [3], or generate actionable mitigations.

Attack trees, introduced by Schneier [1], provide a formal structure for modeling how an adversary achieves a goal through a hierarchy of sub-goals and concrete techniques. When combined with MITRE ATT&CK technique mappings, attack trees become directly actionable: each leaf node maps to a known adversary behavior with documented detection and mitigation strategies. However, constructing these enriched attack trees manually requires deep expertise in both the target system and the threat landscape.

We present THREATFOREST, a multi-agent system that automates the full threat modeling pipeline: from source code repository to MITRE ATT&CK-mapped attack trees with evidence-based mitigations. Our contributions are:

1. **End-to-end pipeline.** A six-stage agent pipeline that takes a code repository as input and produces structured attack trees, TTP mappings, and mitigations, orchestrated as a directed graph with verification gates and retry semantics (§3).
2. **Hybrid TTP mapping.** A two-phase approach combining domain-specific embeddings (ATTACK-BERT) for candidate retrieval with an LLM reviewer for contextual refinement, achieving higher accuracy than either approach alone (§3.4).
3. **Evaluation framework.** A 16-dimension scoring framework with observability infrastructure enabling both automated structural metrics and SME review through Langfuse annotation queues (§4).

2 Related Work

Threat modeling methodologies. STRIDE [2] categorizes threats into six classes (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) and remains the most widely adopted methodology. Attack trees [1] model adversary goals as hierarchical decompositions with AND/OR nodes, enabling quantitative risk analysis. PASTA (Process for Attack Simulation and Threat Analysis) adds a risk-centric perspective. These methodologies provide structure but require manual expertise for every system analyzed.

Tool-assisted threat modeling. Microsoft Threat Modeling Tool [17] generates STRIDE-based threats from data flow diagrams but requires manual diagram construction. OWASP Threat Dragon [16] offers open-source diagramming with threat rule engines. AWS Threat Composer [18] provides guided threat statement authoring with structured templates. None of these tools automate the reasoning required to generate attack trees or map threats to standardized frameworks.

LLM-based threat analysis. ThreatModeling-LLM [8] automates threat identification for banking systems through dataset creation, prompt engineering, and model fine-tuning, achieving promising results on domain-specific threats. Bhatia et al. [9] evaluate whether LLMs can threat model real-world cloud infrastructure, finding that GPT-4.1 and Gemini 2.5 Pro excel at threat identification but with varying performance across prompting strategies. ThreatLens uses retrieval-augmented generation for hardware security verification. These approaches focus on threat *identification* but do not produce structured attack trees or TTP mappings.

MITRE ATT&CK mapping. Automated mapping of security text to ATT&CK techniques has been approached through NLP classification. CVE2ATT&CK [6] uses BERT-based models to map CVE descriptions to ATT&CK techniques. SMET [5] trains ATT&CK BERT using a Siamese network to learn semantic similarity among attack actions, achieving state-of-the-art results on CVE-to-technique mapping. Kim et al. [12] combine ModernBERT with BERTopic for tactic-level classification. These works map *existing* vulnerability descriptions to techniques; THREATFOREST maps *generated* attack tree steps—a distinct task requiring the model to bridge the gap between abstract attack descriptions and concrete ATT&CK techniques.

Domain-specific language models for cybersecurity. SecureBERT [4] adapts BERT to cybersecurity text, improving performance on NER and classification tasks. ATTACK-BERT [7] extends sentence-transformers to produce embeddings specifically for attack action descriptions, enabling semantic similarity search against ATT&CK technique descriptions. We leverage ATTACK-BERT as the retrieval component in our hybrid TTP mapping pipeline (§3.4).

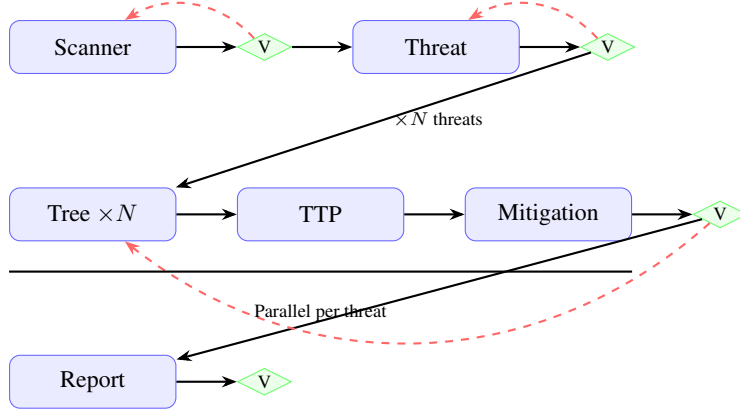


Figure 1: The THREATFOREST pipeline. Boxes are LLM agents; diamonds are deterministic verifiers. Dashed arrows indicate retry edges on verification failure. The tree→TTP→mitigation sub-pipeline runs in parallel for each threat.

Multi-agent systems. The Strands Agents SDK [13] provides a graph-based orchestration framework for multi-agent workflows with tool use, streaming, and telemetry. While multi-agent architectures have been applied to code generation, software testing, and general reasoning, their application to security threat modeling—where outputs must be both technically accurate and structurally valid—remains unexplored.

3 System Architecture

THREATFOREST decomposes threat modeling into six stages, each implemented as an autonomous agent with tool access, orchestrated as a directed graph with verification gates. Figure 1 shows the full pipeline.

3.1 Pipeline Overview and Graph Orchestration

The pipeline is implemented as a Strands Graph [13]—a directed graph where nodes are agent executors and edges carry conditional predicates. Each agent reads its predecessor’s output from a shared state directory (`.threatforest/state/`) and writes structured JSON. This file-based state passing provides three benefits: (1) intermediate outputs are inspectable for debugging, (2) the pipeline is resumable after failures, and (3) parallel agents avoid shared-memory concurrency issues.

The graph defines retry edges: when a verifier detects invalid output, execution flows back to the producing agent for a second attempt, up to a configurable budget of 50 total node executions. This provides self-healing behavior without requiring explicit error handling in each agent.

3.2 Scanner and Threat Generation

Scanner Agent. The scanner receives a repository path and uses two tools: a *structural analyzer* that extracts the project’s file tree, dependency manifests, and infrastructure-as-code definitions; and a *sandboxed file reader* restricted to the repository directory. The agent produces a *project context* containing: cloud provider, technology stack, services, authentication mechanisms, data flows, and deployment model. A deterministic verifier checks that all required fields are present and non-empty.

Threat Agent. The threat agent receives the project context and generates structured threat statements following the pattern:

A [actor] with [access/knowledge], can [action], which leads to [impact], resulting in [consequence].

Each threat includes a title, description, priority, and affected components. The verifier validates JSON structure and ensures each threat references components identified in the scanner context.

3.3 Attack Tree Generation

Attack trees are generated in parallel: one sub-pipeline per threat, all executing concurrently. Each tree agent receives the scanner context and a single threat, producing a hierarchical attack tree where:

- The **root node** represents the adversary’s goal (derived from the threat statement).
- **Intermediate nodes** decompose the goal into sub-goals with AND/OR relationships.
- **Leaf nodes** represent concrete attack techniques.

Each step s_i in the tree is a tuple $(id, title, description, parent_id, is_leaf, category)$ where $category \in \{\text{fact, action, detection, gate}\}$. The tree structure enables systematic enumeration of attack paths: each root-to-leaf path represents a complete attack scenario.

For a system with N threats, the parallel fan-out produces N independent sub-pipelines (tree $\rightarrow$ TTP $\rightarrow$ mitigation), providing approximately $N \times$ speedup over sequential execution.

3.4 TTP Mapping: Hybrid Embedding + LLM

Mapping attack tree steps to MITRE ATT&CK techniques is the most technically challenging stage. We employ a two-phase hybrid approach that combines the efficiency of embedding-based retrieval with the contextual reasoning of an LLM.

Phase 1: Embedding Retrieval. Each attack step description d_i is encoded using ATTACK-BERT [7], a sentence-transformer fine-tuned on cybersecurity text. We compute cosine similarity against pre-encoded ATT&CK technique descriptions $\{t_1, \dots, t_M\}$ where M is the number of techniques in the ATT&CK matrix:

$$\text{sim}(d_i, t_j) = \frac{\mathbf{e}(d_i) \cdot \mathbf{e}(t_j)}{\|\mathbf{e}(d_i)\| \|\mathbf{e}(t_j)\|} \quad (1)$$

where $\mathbf{e}(\cdot)$ denotes the ATTACK-BERT embedding function. For each step d_i , we retrieve the top- K candidates ranked by similarity:

$$\text{candidates}(d_i) = \text{top-}K(\{\text{sim}(d_i, t_j) \mid j = 1, \dots, M\}) \quad (2)$$

We use $K = 5$ and a minimum similarity threshold $\tau = 0.3$. The top-1 candidate is selected as the initial mapping.

Phase 2: LLM Reviewer. The embedding-based top-1 mapping is often correct for well-described attack steps but fails when surface-level textual similarity diverges from semantic intent. For example, a step describing “credential theft via compromised IAM role” may match “T1557.004 Evil Twin” (a Wi-Fi attack) based on shared vocabulary around “compromise” rather than the semantically correct “T1078.004 Valid Accounts: Cloud Accounts.”

The LLM reviewer agent receives the top-1 summary and has access to a `get_ttp_alternatives` tool that returns the full top- K candidates for any step. The reviewer:

1. Examines each top-1 mapping for semantic correctness.
2. For suspicious mappings, queries alternatives and selects the best match.
3. Records whether it overrode the embedding’s top-1 and provides reasoning.

This hybrid approach leverages the embedding model’s efficiency (encoding all steps in a single batch) while using the LLM only for refinement, keeping costs proportional to the number of overrides rather than the total number of steps.

3.5 Mitigation and Verification

Mitigation Agent. The mitigation agent receives the attack trees, TTP mappings, and scanner context, synthesizing per-technique mitigations. Each mitigation includes: the specific AWS service or configuration to implement, the attack steps it addresses, and evidence linking the mitigation to the

identified technique. This evidence-based approach ensures mitigations are actionable rather than generic.

Verifier Pattern. Every stage in the pipeline includes a deterministic verifier—a pure function (no LLM) that validates structural properties of the output:

- **Scanner verifier:** All required context fields present and non-empty.
- **Threat verifier:** Valid JSON, each threat has required fields, references valid components.
- **Pipeline verifier:** Attack trees have valid parent-child relationships, TTP mappings reference existing steps, mitigations reference existing techniques.
- **Report verifier:** Output file exists with expected sections.

On verification failure, the graph’s retry edge routes execution back to the producing agent. This pattern provides quality gates without the cost or non-determinism of LLM-based validation.

Report Generator. The final stage is deterministic (no LLM): it compiles all state files into a structured Markdown report and an interactive HTML dashboard with attack tree visualizations.

4 Evaluation Methodology

We design an evaluation framework that combines automated structural metrics with human expert assessment, supported by an observability infrastructure that captures every agent interaction.

4.1 Scoring Dimensions

We define 16 scoring dimensions across four capabilities, each using a 5-point categorical scale $C = \{\text{excellent, good, acceptable, poor, unacceptable}\}$ mapped to numeric values $\{1.0, 0.75, 0.5, 0.25, 0.0\}$:

Threat Statement Quality (5 dimensions): overall quality, relevance to application context, completeness of threat coverage, technical accuracy, and hallucination detection.

Attack Tree Quality (6 dimensions): overall quality, structural quality (depth, branching, organization), technical realism of attack techniques, logical progression of attack paths, completeness of attack vector coverage, and actionability for defenders.

TTP Mapping Quality (1 dimension): correctness of MITRE ATT&CK technique assignment, using a specialized scale replacing “unacceptable” with “no_mapping” for steps where no technique applies.

Mitigation Quality (4 dimensions): actionability (concrete implementable steps), specificity to the identified threat and technology stack, coverage of identified attack paths, and technical accuracy of recommended controls.

4.2 Observability Infrastructure

THREATFOREST produces two complementary trace types:

OTEL traces (automatic). The Strands SDK [13] emits OpenTelemetry spans for every LLM call, tool invocation, and agent event loop cycle. These are exported to Langfuse [14] via the OTLP protocol, capturing token counts, latencies, model parameters, and full input/output content. All traces within a pipeline run share a session identifier for grouping.

Annotation traces (structured). At each subgraph boundary (scanner, threat generation, attack tree generation, TTP mapping, mitigation generation), we push a Langfuse SDK trace with clean input/output JSON. These traces are tagged with `annotation` for routing to SME review queues.

4.3 SME Review Pipeline

Annotation traces are routed to Langfuse annotation queues—one per capability. Each queue presents the reviewer with the agent’s input context and generated output, along with the applicable scoring

Table 1: Pipeline output summary across application domains.

Domain	Threats	Trees	Steps	TTPs	Unique	Mitigations
S3 Multi-Tenant SaaS	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Healthcare (HCLS)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
IoT Device Mgmt	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Vehicle Platform	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
[TODO: More domains]						
Average	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

dimensions. Reviewers assign categorical scores independently for each dimension and may add free-text feedback.

4.4 TTP Ground Truth Dataset

Individual TTP mappings are automatically pushed as Langfuse dataset items with the attack step description as input and the mapped technique as metadata. The `expected_output` field is initially null; SMEs label each mapping as correct or incorrect, building a ground truth corpus over time. This corpus serves dual purposes: (1) evaluating embedding retrieval accuracy via $\text{precision@}K$, and (2) providing training pairs for fine-tuning domain-specific embedding models.

4.5 Automated Structural Metrics

We compute the following metrics automatically from pipeline outputs:

- **Tree depth:** Maximum root-to-leaf path length per attack tree.
- **Step count:** Total attack steps across all trees.
- **Technique diversity:** $|\text{unique techniques}|/|\text{total mappings}|$, measuring how broadly the system maps across ATT&CK.
- **Phase coverage:** Fraction of ATT&CK tactics (Initial Access through Impact) represented in the generated mappings.
- **Reviewer override rate:** Fraction of TTP mappings where the LLM reviewer changed the embedding’s top-1 selection.

5 Results

We evaluate THREATFOREST on [TODO: N] sample applications spanning diverse cloud architectures. All experiments use Claude Opus 4.6 via AWS Bedrock with ATTACK-BERT for TTP embedding retrieval.

5.1 Pipeline Output Summary

Table 1 summarizes the pipeline outputs across domains.

5.2 Structural Quality Metrics

Table 2 reports automated structural metrics.

5.3 TTP Mapping Analysis

Table 3 analyzes the hybrid TTP mapping approach.

The override rate measures how often the LLM reviewer changed the embedding’s top-1 selection. A high override rate indicates the embedding model struggles with the domain; a low rate with high post-review accuracy suggests the embedding is reliable and the reviewer adds marginal value.

Table 2: Structural quality metrics across domains.

Domain	Avg Depth	Avg Steps/Tree	Technique Diversity	Phase Coverage
S3 Multi-Tenant SaaS	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Healthcare (HCLS)	[TODO:]	[TODO:]	[TODO:]	[TODO:]
IoT Device Mgmt	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Vehicle Platform	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Average	[TODO:]	[TODO:]	[TODO:]	[TODO:]

Table 3: TTP mapping analysis. Embedding accuracy is top-1 precision before LLM review; post-review accuracy is measured against SME labels.

Domain	Total Mappings	Override Rate	Embedding Acc.	Post-Review Acc.
S3 Multi-Tenant SaaS	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Healthcare (HCLS)	[TODO:]	[TODO:]	[TODO:]	[TODO:]
IoT Device Mgmt	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Vehicle Platform	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Average	[TODO:]	[TODO:]	[TODO:]	[TODO:]

5.4 SME Evaluation Scores

[TODO: Table with SME scores per dimension, per domain. Median and IQR across reviewers. Include inter-rater reliability (Cohen’s κ or Krippendorff’s α).]

6 Discussion and Conclusion

6.1 Design Tradeoffs

Parallel vs. sequential execution. The parallel fan-out for attack tree generation provides significant speedup but prevents inter-threat context sharing. A sequential approach could identify cross-cutting attack paths (e.g., a compromised IAM role enabling multiple threats), but at $N \times$ cost. We chose parallelism as the default, noting that cross-threat analysis could be added as a post-processing consolidation step.

Hybrid TTP mapping. Pure embedding retrieval is fast and deterministic but lacks contextual reasoning. Pure LLM mapping avoids the embedding model entirely but is expensive (one LLM call per step) and prone to hallucinating non-existent technique IDs. Our hybrid approach uses the embedding for efficient candidate retrieval and the LLM only for refinement, keeping costs proportional to the override rate rather than the total step count.

File-based state passing. Agents communicate through JSON files rather than in-memory message passing. This adds I/O overhead but provides debuggability (every intermediate output is inspectable), resumability (a failed pipeline can restart from the last successful stage), and simplicity in parallel execution (no shared-memory synchronization).

6.2 Limitations

Evaluation scope. Our evaluation uses sample applications rather than production systems. While these applications represent diverse cloud architectures, they may not capture the full complexity of real-world deployments with legacy components, multi-cloud configurations, or custom middleware.

Model dependence. All experiments use a single model family (Claude via AWS Bedrock). Performance may vary across model providers, and the system’s effectiveness is bounded by the underlying model’s security knowledge.

SME availability. The evaluation framework requires security experts for scoring, which limits the scale of evaluation. [TODO: Report actual SME pool size and inter-rater reliability.]

Table 4: SME evaluation scores (median) across domains. Scale: 1.0 = Excellent, 0.0 = Unacceptable.

Dimension	S3	HCLS	IoT	Vehicle
<i>Threat Statements</i>				
Overall Quality	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Relevance	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Completeness	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Technical Accuracy	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Hallucination	[TODO:]	[TODO:]	[TODO:]	[TODO:]
<i>Attack Trees</i>				
Overall Quality	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Structural Quality	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Technical Realism	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Attack Path Logic	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Completeness	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Actionability	[TODO:]	[TODO:]	[TODO:]	[TODO:]
<i>TTP Mapping</i>				
Mapping Quality	[TODO:]	[TODO:]	[TODO:]	[TODO:]
<i>Mitigations</i>				
Actionability	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Specificity	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Coverage	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Technical Accuracy	[TODO:]	[TODO:]	[TODO:]	[TODO:]

ATT&CK coverage. MITRE ATT&CK, while comprehensive, has known gaps in cloud-native, container, and serverless attack techniques. Mappings to these domains may be less precise.

6.3 Future Work

Automated attack matrix ingestion. We plan to extend beyond MITRE ATT&CK by ingesting additional threat frameworks—the Wiz cloud threat matrix [19], AWS Threat Technique Catalog (TTC), OWASP attack patterns, and CAPEC [20]. A unified STIX-based knowledge graph would normalize techniques across matrices, increasing coverage for cloud-native attack surfaces that MITRE alone underrepresents.

Domain-specific embedding fine-tuning. The TTP ground truth dataset built through our evaluation framework (§4) provides SME-labeled correct/incorrect mappings. These labels can be used to fine-tune a custom security embedding model starting from ATTACK-BERT, improving retrieval accuracy for the specific threat domains THREATFOREST encounters and reducing the LLM reviewer’s override rate over time.

Prompt optimization. The SME-scored traces captured by our observability infrastructure are directly compatible with DSPy’s MIPROv2 optimizer [11], which can jointly optimize instructions and few-shot demonstrations for multi-stage LM programs. Each scored trace provides a training signal for the corresponding pipeline stage.

Automated evaluation. To reduce dependence on SME availability, we plan to develop LLM-as-a-judge evaluators calibrated against the SME-scored ground truth, enabling continuous evaluation at scale.

6.4 Conclusion

We presented THREATFOREST, a multi-agent system that automates threat modeling from source code repositories to MITRE ATT&CK-mapped attack trees with evidence-based mitigations. Our hybrid TTP mapping approach combines domain-specific embeddings with LLM refinement, and our verifier pattern provides quality gates without LLM cost. The accompanying evaluation framework with 16 scoring dimensions and Langfuse-based observability enables systematic measurement of threat modeling quality. **[TODO: Summarize key results.]** To our knowledge, this is the first

end-to-end automated threat modeling system with structured evaluation, establishing baselines against which future systems can be measured.

References

- [1] B. Schneier. Attack trees. *Dr. Dobbs's Journal*, 24(12):21–29, 1999.
- [2] A. Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, 2014.
- [3] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas. MITRE ATT&CK: Design and philosophy. Technical report, The MITRE Corporation, 2018.
- [4] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer. SecureBERT: A domain-specific language model for cybersecurity. In *Intl. Conf. on Security and Privacy in Communication Systems*, 2023.
- [5] A. Kuppia, L. Aouad, and N.-A. Le-Khac. SMET: Semantic mapping of CVE to ATT&CK and its application to cyber security. In *DBSec*, 2023.
- [6] A. Grigorescu, V. Iorga, C. Sandescu, and R. Rughinis. CVE2ATT&CK: BERT-based mapping of CVEs to MITRE ATT&CK techniques. *Algorithms*, 15(9):314, 2022.
- [7] Basel. ATT&CK BERT: A cybersecurity domain-specific sentence-transformer. <https://huggingface.co/basel/ATTACK-BERT>, 2024.
- [8] M. Alam, S. Iqbal, and M. Aleem. Automating threat modeling using large language models for banking systems. *arXiv preprint arXiv:2411.17058*, 2024.
- [9] A. Bhatia, A. Singhal, and S. Jha. Can LLMs threat model real-world cloud infrastructure? *arXiv preprint arXiv:2505.11565*, 2025.
- [10] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *ICLR*, 2024.
- [11] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Li, et al. Optimizing instructions and demonstrations for multi-stage language model programs. *arXiv preprint arXiv:2406.11695*, 2024.
- [12] J. Kim, S. Lee, and H. Park. Improving automatic classification of MITRE ATT&CK tactics using NLP-based ModernBERT and BERTopic. *Electronics*, 14(22):4434, 2025.
- [13] AWS. Strands Agents: A model-driven approach to building AI agents. <https://strandsagents.com>, 2025.
- [14] Langfuse. Open-source LLM engineering platform. <https://langfuse.com>, 2024.
- [15] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *EMNLP*, 2019.
- [16] OWASP. OWASP Threat Dragon. <https://owasp.org/www-project-threat-dragon/>, 2023.
- [17] Microsoft. Microsoft Threat Modeling Tool. <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool>, 2023.
- [18] AWS. AWS Threat Composer. <https://github.com/aws-labs/threat-composer>, 2024.
- [19] Wiz. Cloud threat landscape. <https://threats.wiz.io/>, 2024.
- [20] MITRE. CAPEC: Common attack pattern enumeration and classification. <https://capec.mitre.org/>, 2024.

A Score Definitions Reference

Table 5 lists all 16 score definitions registered with Langfuse.

B Cosine Similarity for TTP Retrieval

Given an attack step description d_i and ATT&CK technique description t_j , ATTACK-BERT produces embedding vectors $\mathbf{e}(d_i), \mathbf{e}(t_j) \in \mathbb{R}^{768}$. The cosine similarity is:

Table 5: Complete score definitions. All use the 5-point categorical scale unless noted.

Prefix	Name	Description
threat_	overall_quality	Holistic assessment of generated threats
	relevance_to_context	Match to application context
	completeness	Coverage of threat categories
	technical_accuracy	Technical correctness
	hallucination_score	Absence of fabricated content
attack_tree_	overall_quality	Holistic assessment of the attack tree
	structural_quality	Depth, branching, organization
	technical_realism	Feasibility of attack techniques
	attack_path_logic	Logical progression from access to impact
	completeness	Coverage of attack vectors and phases
ttp_	actionability	Usefulness for defenders
	mapping_quality	Quality of MITRE ATT&CK technique mapping*
mitigation_	actionability	Concrete, implementable steps
	specificity	Tailored to identified threat and tech stack
	coverage	Addresses all identified attack paths
	technical_accuracy	Correctness of recommended controls

*Uses specialized scale: excellent, good, acceptable, poor, no_mapping.

$$\text{sim}(d_i, t_j) = \frac{\sum_{k=1}^{768} e_k(d_i) \cdot e_k(t_j)}{\sqrt{\sum_{k=1}^{768} e_k(d_i)^2} \cdot \sqrt{\sum_{k=1}^{768} e_k(t_j)^2}} \quad (3)$$

The top- K candidates are retrieved by sorting all M techniques by descending similarity and selecting the first K entries above threshold τ :

$$\text{candidates}(d_i) = \{t_j \mid \text{sim}(d_i, t_j) \geq \tau\} \text{ sorted by } \text{sim}(d_i, t_j) \text{ desc, truncated to } K \quad (4)$$

In our experiments, $K = 5$ and $\tau = 0.3$.